# ELEC50010 INSTRUCTION ARCHITECTURES AND COMPILERS

*NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–8 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.*

1. Use the instruction set summary provided, translate the following RISC-V RV32I assembly language instructions into machine code instructions as 32-bit hexadecimal numbers.

   a) `sltiu s0, t1, 255` [ 4 ]

   b) `auipc a2, 0xF3A14` [ 4 ]

   *Answer:*
   *This question tests a student's understanding of how instructions are encoded.*

   - *`32'hff33413`*
   - *`32'hf3a14617`*

2. What are the assembly language instructions of the following RISC-V RV32I machine code instructions? Briefly explain what each instruction does.

   a) `32'h4FF91E03` [ 4 ]

   b) `32'h40535513` [ 4 ]

   *Answer:*
   *This question tests a student's understanding of how to decode instruction from the machine code to assembly instructions, and the operation of the two instructions in the question.*

   - *`add t0,t1,zero`*
   - *`sw a0,10(sp)`*

Answers

3. a) It is known that $1 + x^2 + x^3 + x^4 + x^8$ is a primitive polynomial. Design in SystemVerilog an 8-bit pseudo-random binary sequence generator module with the interface defined in Figure 3.1. [ 6 ]

b) Assuming that the sequence starts with the value `8'h02`, write down the first five values of the sequence. [ 4 ]

```
module pseudo8 (
    input logic          rst,    // reset to initial state
    input logic          clk,    // clock input
    output logic [7:0]   random  // 8-bit pseudo random number
);
```

Figure 3.1

*Answer:*
*This question tests student's understanding of using shift register to implement a pseudorandom binary sequence generator using primitive polynomial in SystemVerilog.*

- *Here is a typical SystemVerilog design of the 8-bit PRBS generator:*

```
module pseudo8 (
    input logic          rst,    // reset to initial state
    input logic          clk,    // clock input
    output logic [7:0]   random  // 8-bit pseudo random number
);

    logic [8:1]     sreg;        // internal shift register

    always_ff @(posedge clk)
        if (rst)
            sreg <= 8'b1;
        else
            sreg <= {sreg[7:1], sreg[2]^sreg[3]^sreg[4]^sreg[8]};

    assign random = sreg;
endmodule
```

Figure 3.2

- *The sequence is: `8'h02, 8'h05, 8'h0B, 8'h16, 8'h2C`, or in binary:*

| Binary | Hex |
|---|---|
| 0000 0010 | 0x02 |
| 0000 0101 | 0x05 |
| 0000 1011 | 0x0B |
| 0001 0110 | 0x16 |
| 0010 1100 | 0x2C |

Figure 3.3

Answers

4. Figure 4.1 shows an implementation of the RISC-V RV32I processor using five stages of pipelining when executing the five instructions on the left. The register file is designed in such a way that when a register value is updated, it is only available on the following cycle.

a) For the five instructions shown in Figure 4.1, identify where and when data hazards occur. To help you show your answer, a duplicate copy of Figure 4.1 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. State any assumptions made. [ 4 ]

b) Assume that there is no hazard unit included in the microarchitecture, insert the **minimum** number of NOP instructions to the diagram so that all hazards are mitigated. [ 4 ]

c) A hazard unit can be used to mitigate the hazards in this snippet of code. For each of the hazard cases identified above, explain what operations the hazard unit must perform to mitigate them. [ 4 ]

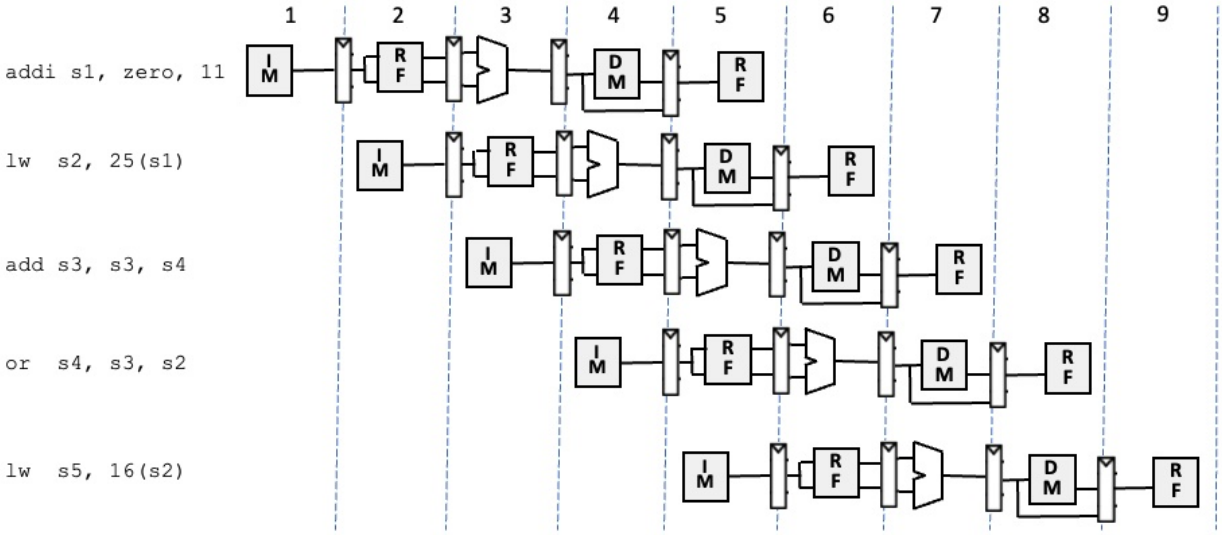


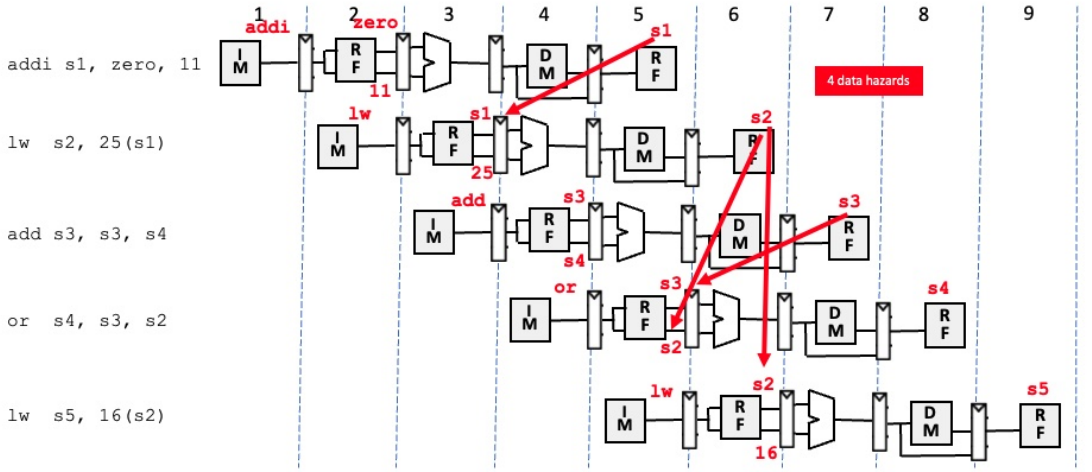

Figure 4.1

Answers

*Answer:*
*This question tests student's grasp of data hazard in a pipelined architecture, and how this may be mitigated.*

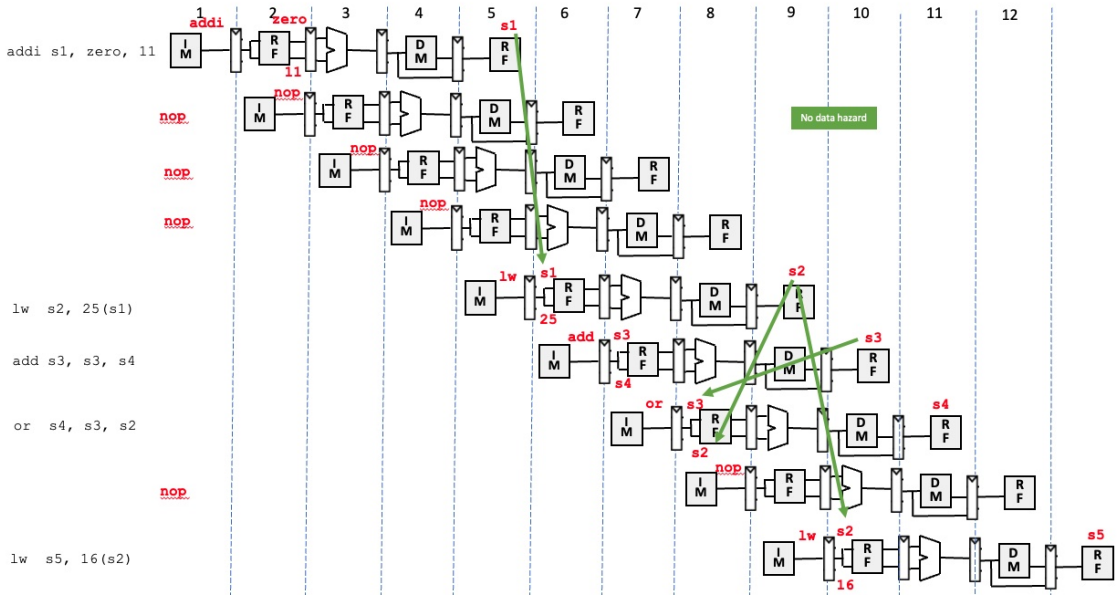
*(a) See the following diagram.*

*The data hazard of s1 in cycle 3, s2 in cycle 5 and s3 in cycle 5 are obvious. However, s2 in cycle 6 being a data hazard is less obvious. This is because on cycle 6, s2 is only available at the end of the cycle (after it is being written to RF) while it is also required by instruction 5 at the start of the cycle. Therefore this too is a data hazard! Hence there are four data hazards in total.*




*[ 4 ]*

*(b) Here are where the NOPs are required. Students are not expected to draw the pipeline diagrams, but should insert the NOPs in the correct places. Green arrows show why data hazards are avoided.*



*[ 4 ]*

*(c) In all four cases, mitigation is by forwarding or bypassing pipeline registers.*

** Hazard 1: s1 needed by 2nd instruction in cycle 3, but is only written to RF at the end of cycle 5. Mitigation: Since s1 is known at end of cycle 3, and is only used by 2nd instruction at start of cycle 4, we can mitigate this using BYPASSING or FORWARDING. * Hazard 2: s2 written to RF at the end of cycle 6, but is read by instruction 4 in cycle 5. Mitigation: s2 is read from DM at end cycle 5, therefore it can also be forward for use at start of cycle 6.*

** Hazard 3: s3 is required for the OR operation but is only written to RF in cycle 7. Mitigation: s3 is available at output of ALU. Use Bypassing.*

** Hazard 4: s2 is written to and read from simultaneously in cycle 6 by 2nd and 4th instructions. Mitigation: Instead of reading from RF, bypass it and take s2 from data input.* *[ 4 ]*

Answers

5. A RISC-V RV32I processor has an 8-entry data cache that can be configured as direct mapped cache with different block sizes, and each entry stores a 32-bit data word. The assembly program shown in Figure 5.1 is executed by the processor.

a) The data cache, depicted in Figure 5.2, is organised as single word direct cache and it is initially empty.

- How many bits are in the V, tag and data fields of the cache? [ 2 ]
- Show the contents of the cache on Figure 5.2 after finishing executing the code in Figure 5.1. To help you show your answer, a duplicate copy of Figure 5.2 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. [ 3 ]
- What is the miss rate? [ 2 ]

b) The data cache is reconfigured into a 2-entry direct mapped cache, where each entry stores four 32-bit words and the cache is initially empty.

- Show the contents of the cache after finishing executing the code in Figure 5.1. [ 3 ]
- What is the miss rate for this cache configuration? [ 2 ]

```
1          addi    s0, zero, 4
2          addi    s1, zero, 0x0
3          addi    s2, zero, 0x20
4   LOOP:  beq     s0, zero, DONE
5          lw      t1, 0(s1)
6          lw      t2, 8(s1)
7          lw      t3, 4(s2)
8          lw      t4, 8(s2)
9          addi    s0, s0, -1
10         j       LOOP
11  DONE:
```

Figure 5.1

| V | tag | data | set |
|---|---|---|---|
| | | | 111 |
| | | | 110 |
| | | | 101 |
| | | | 100 |
| | | | 011 |
| | | | 010 |
| | | | 001 |
| | | | 000 |

Figure 5.2

Answers

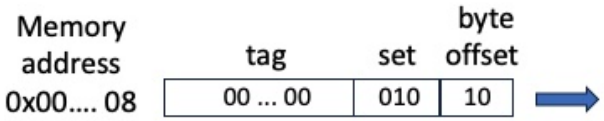
*Answer:*

*This question tests student's understanding of how cache is organised, addressed and utilized.*

*(a)*

*(i) This is direct mapped cache with 8 entries. Therefore, memory address LSB 2-bit are byte offset, next 3 bits are for set within cache. Hence the tag bit must be 32-5 = 27 bits. V (valid) is 1 bit, and data is 32-bits.* *[ 2 ]*

*(ii) The contents of the cache is shown in the diagram below:*




| V | $tag_{27}$ | $data_{32}$ | set |
|---|---|---|---|
| 0 | | | 111 |
| 1 | 00 ... 01 | mem(0x00 ... 28) | 110 |
| 1 | 00 ... 01 | mem(0x00 ... 24) | 101 |
| 0 | | | 100 |
| 0 | | | 011 |
| 1 | 00 ... 00 | mem(0x00 ... 08) | 010 |
| 0 | | | 001 |
| 1 | 00 ... 00 | mem(0x00 ... 00) | 000 |

*[ 3 ]*

*(iii) The program goes around the loop four times. The first time, all four memory accesses are misses. Thereafter, they are all hits. Therefore, total number of accesses are: 4x4 = 16, with 4 misses, miss rate is 25%.* *[ 2 ]*

*(b)*

*(i) The contents of the cache is shown in the diagram below:*

| V | $tag_{29}$ | $data_11_{32}$ | $data_10_{32}$ | $data_01_{32}$ | $data_00_{32}$ | set |
|---|---|---|---|---|---|---|
| 1 | 00 ... 04 | | mem(0x00 ... 28) | mem(0x00 ... 24) | | 1 |
| 1 | 00 ... 00 | | mem(0x00 ... 08) | | mem(0x00 ... 00) | 0 |

*[ 3 ]*

*(ii) Fetch from memory address 0x0 and 0x24 also fetches contents from 0x8 and 0x28 respectively. Therefore there are now only two misses out of 16 access. Miss rate is now only 12.5%.* *[ 2 ]*

Answers

6. Explain how `break` statements are handled by a C compiler. *[Hint: note that* `break` *statements may be compiled differently depending on where they appear in the source code.]* [ 6 ]

*Answer:*
*The* `break` *statement is compiled to an unconditional jump. The target of the jump is a label that is placed immediately after the innermost loop or switch statement that contains the* `break`*. If the* `break` *statement is not inside a loop or switch statement, then this is a programmer fault, and the compiler should generate an error message.*

Answers

7. Nathan suggests a compiler optimisation that detects the code pattern

```
for (i=0; i<N; i++)
  CODE
```

(where `N` is an integer constant and `CODE` is a block of code that does not mention `i`) and replaces it with

```
CODE
CODE
...
CODE
```

} `N` times.

a) Explain how transforming the code in this way could *improve* the performance of the compiled program. [ 6 ]

*Answer:*
*Here are three possible answers:*

- *The transformed code requires fewer conditional branches, which means it doesn't have to evaluate the condition* `i<N` *each time* `CODE` *is executed.*
- *Removing conditional branches also avoids the risk of pipeline stalls associated with mispredicted branches.*
- *The simple straight-line code that results from this transformation may be easier to optimise further, e.g. with scheduling or peephole optimisations.*

b) Explain how transforming the code in this way could *reduce* the performance of the compiled program. [ 6 ]

*Answer:*
*Here is one possible answer:*

- *If* `N` *is very large then the generated code could contain a lot of repeated instructions, meaning that the final executable is large, and hence slow to load into memory when the program starts up.*

c) Are there any conditions on `N` or `CODE` that must hold in order for Nathan's proposed transformation to be valid? [ 2 ]

*Answer:*
*Here are three possible answers:*

- `N` *should not be negative.*
- `CODE` *should not contain any* `break` *or* `continue` *statements.*
- `CODE` *should not mention any variable that aliases with* `i`*. For instance, if* `int *j = &i` *occurs before the loop, then* `CODE` *should not mention* `*j`*. The practical way to avoid this situation is to require that* `i` *never has its address taken.*

8. This question is about building a parser for the following context-free grammar.

$$L ::= L\texttt{aa} \mid \texttt{b}$$

a) Is *L nullable*? Briefly explain your answer. [ 2 ]

Answers

*Answer:*
*No, because it cannot generate the empty word.*

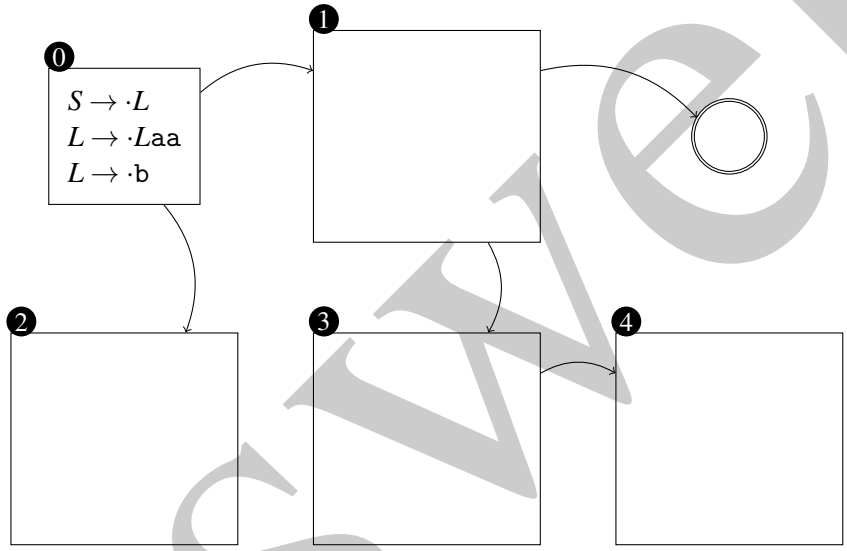
b) What is the *FIRST* set of $L$? Briefly explain your answer. [ 3 ]

*Answer:*
$\{\texttt{b}\}$, *because every word generated by L begins with a* `b`.

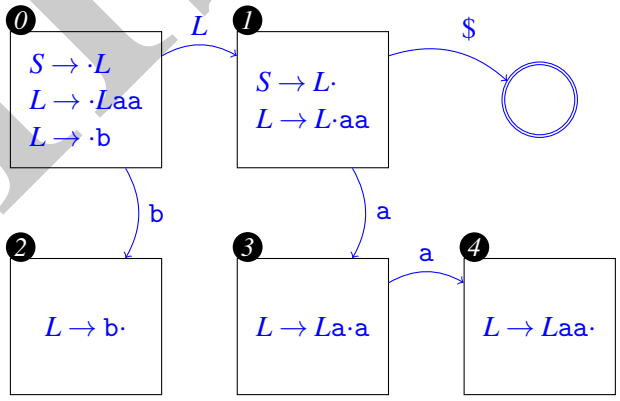
c) What is the *FOLLOW* set of $L$? Briefly explain your answer. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [ 3 ]

*Answer:*
$\{\texttt{a}, \$\}$, *because every word generated by L is followed either by an* `a` *or the end of the input.*

d) Suppose we decide to build a shift/reduce parser for $L$. Complete the following diagram by filling the four empty rectangles with item-sets and labelling each edge with a symbol. The diagram is reproduced at the end of this paper so you can write on it, tear it off, and submit it with your answers. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [ 9 ]




*Answer:*




e) Using information from your answers to parts (a)–(d), complete the following parsing table for the shift/reduce parser. The table is reproduced at the end of this paper so you can write on it, tear it off, and submit it with your answers.

Answers

| STATE | ACTION | | | GOTO |
|---|---|---|---|---|
| | a | b | $ | *L* |
| ❶ | | | | |
| ❶ | | | | |
| ❷ | | | | |
| ❸ | | | | |
| ❹ | | | | |

*[Hint: Each cell in the* **ACTION** *columns should either:*

- *be left empty,*
- *contain 'accept',*
- *contain 'sN' where N is the state to enter after shifting, or*
- *contain 'r(RULE)' where RULE is the grammar rule that should be reduced,*

*and each cell in the* **GOTO** *column should either:*

- *be left empty, or*
- *contain 'N' where N is the state to enter after reducing.]*

[ 9 ]

*Answer:*

| STATE | ACTION | | | GOTO |
|---|---|---|---|---|
| | a | b | $ | *L* |
| ❶ | | *s2* | | *1* |
| ❶ | *s3* | | *accept* | |
| ❷ | *r*(*L* → b) | | *r*(*L* → b) | |
| ❸ | *s4* | | | |
| ❹ | *r*(*L* → *L*aa) | | *r*(*L* → *L*aa) | |

f) Explain why the grammar given at the start of this question is not suitable for top-down parsing, and show how it could be rewritten so that it would become suitable. [ 4 ]

*Answer:*
*It's not suitable because it is left-recursive. It could be rewritten as follows:*

$$L ::= \texttt{b}M$$
$$M ::= \texttt{aa}M \mid \varepsilon$$

Answers

# RISC-V Instruction Set Summary (1)

## Register (R-type) Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| R-type | funct7 | | | | | | | rs2 | | | | | rs1 | | | | | funct3 | | | rd | | | | | opcode | | | | | | |

| opcode | funct3 | funct7 | instruction | description | operation |
|---|---|---|---|---|---|
| 7'h33 | 3'b000 | 7'h00 | add rd, rs1, rs2 | add | rd = rs1 + rs2 |
| 7'h33 | 3'b000 | 7'h20 | sub rd, rs1, rs2 | subtract | rd = rs1 - rs2 |
| 7'h33 | 3'b001 | 7'h00 | sll rd, rs1, rs2 | shift left logical | rd = rs1 << $rs2_{4:0}$ |
| 7'h33 | 3'b010 | 7'h00 | slt rd, rs1, rs2 | set less than | rd = (rs1 < rs2) |
| 7'h33 | 3'b011 | 7'h00 | sltu rd, rs1, rs2 | set less than unsigned | rd = (rs1 < rs2)$^{unsigned}$ |
| 7'h33 | 3'b100 | 7'h00 | xor rd, rs1, rs2 | bit-wise XOR | rd = rs1 ^ rs2 |
| 7'h33 | 3'b101 | 7'h00 | srl rd, rs1, rs2 | shfit right logical | rd = rs1 >> $rs2_{4:0}$ |
| 7'h33 | 3'b101 | 7'h20 | sra rd, rs1, rs2 | shfit right arithmetic | rd = rs1 >>> $rs2_{4:0}$ |
| 7'h33 | 3'b110 | 7'h00 | or rd, rs1, rs2 | bit-wise OR | rd = rs1 \| rs2 |
| 7'h33 | 3'b111 | 7'h00 | and rd, rs1, rs2 | bit-wise AND | rd = rs1 & rs2 |

## Immediate (I-type) Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I-type | imm[11:0] | | | | | | | | | | | | rs1 | | | | | funct3 | | | rd | | | | | opcode | | | | | | |

| opcode | funct3 | imm[11:5] | instruction | description | operation |
|---|---|---|---|---|---|
| 7'h03 | 3'b000 | - | `lb rd, imm(rs1)` | load byte immediate | rd = sign_ext ([address]$_{7:0}$) |
| 7'h03 | 3'b001 | - | `lh rd, imm(rs1)` | load half immediate | rd = sign_ext ([address]$_{15:0}$) |
| 7'h03 | 3'b010 | - | `lw rd, imm(rs1)` | load word immediate | rd = [address]$_{31:0}$ |
| 7'h03 | 3'b100 | - | `lbu rd, imm(rs1)` | load byte unsigned | rd = zerp_ext ([address]$_{7:0}$) |
| 7'h03 | 3'b101 | - | `lhu rd, imm(rs1)` | load half unsigned | rd = zero_ext ([address]$_{15:0}$) |
| 7'h13 | 3'b000 | - | `addi rd, rs1, imm` | add immediate | rd = rs1 + sign_ext (imm) |
| 7'h13 | 3'b001 | 7'h00 | `slli rd, rs1, uimm` | Shift left logical immediate | rd = rs1 << uimm |
| 7'h13 | 3'b010 | - | `slti rd, rs1, imm` | set less than immediate | rd = (rs1 < sign_ext(imm)) |
| 7'h13 | 3'b011 | - | `sltiu rd, rs1, uimm` | set less than imm. unsigned | rd = (rs1 < zero_ext(imm)) |
| 7'h13 | 3'b100 | - | `xori rd, rs1, imm` | bit-wise XOR immediate | rd = rs1 ^ sign_ext (imm) |
| 7'h13 | 3'b101 | 7'h00 | `srli rd, rs1, uimm` | Shift right logical immediate | rd = rs1 >> uimm |
| 7'h13 | 3'b101 | 7'h20 | `srai rd, rs1, uimm` | Shift right arithmetic imm. | rd = rs1 >> uimm |
| 7'h13 | 3'b110 | - | `ori rd, rs1, imm` | bit-wise OR immediate | rd = rs1 \| sign_ext (imm) |
| 7'h13 | 3'b111 | - | `andi rd, rs1, imm` | bit-wise AND immediate | rd = rs1 & sign_ext (imm) |

## Upper Immediate (U-type) Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| U-type | imm[31:12] | | | | | | | | | | | | | | | | | | | | rd | | | | | opcode | | | | | | |

| opcode | instruction | description | operation |
|---|---|---|---|
| 7'h17 | `auipc rd, upimm` | add upper immediate to PC | rd = {upimm, 12'b0} + PC |
| 7'h37 | `lui rd, upimm` | load upper immediate | rd = {upimm, 12'b0} |

# RISC-V Instruction Set Summary (2)

## Store (S-type) Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| S-type | imm[11:5] | | | | | | | rs2 | | | | | rs1 | | | | | funct3 | | | imm[4:0] | | | | | opcode | | | | | | |

| opcode | funct3 | instruction | description | operation |
|---|---|---|---|---|
| 7'h23 | 3'b000 | `sb rs2, imm(rs1)` | store byte immediate | $[address]_{7:0} = rs2_{7:0}$ |
| 7'h23 | 3'b001 | `sh rs2, imm(rs1)` | store half immediate | $[address]_{15:0} = rs2_{15:0}$ |
| 7'h23 | 3'b010 | `sw rs2, imm(rs1)` | store word immediate | $[address]_{31:0} = rs2$ |

## Branch (B-type) Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| B-type | [12] | imm[10:5] | | | | | | rs2 | | | | | rs1 | | | | | funct3 | | | imm[4:1] | | | | [11] | opcode | | | | | | |

| opcode | funct3 | instruction | description | operation |
|---|---|---|---|---|
| 7'h63 | 3'b000 | `beq rs1, rs2, label` | branch if equal | If (rs1 == rs2) PC = BTA |
| 7'h63 | 3'b001 | `bne rs1, rs2, label` | branch if not equal | If (rs1 ≠ rs2) PC = BTA |
| 7'h63 | 3'b100 | `blt rs1, rs2, label` | branch if less than | If (rs1 < rs2) PC = BTA |
| 7'h63 | 3'b101 | `bge rs1, rs2, label` | branch if greater than or equal | If (rs1 ≥ rs2) PC = BTA |
| 7'h63 | 3'b110 | `bltu rs1, rs2, label` | branch if less than unsigned | If (rs1 < rs2) PC = BTA |
| 7'h63 | 3'b111 | `bgeu rs1, rs2, label` | branch if greater or equal unsigned | If (rs1 ≥ rs2) PC = BTA |

BTA = Branch Target Address, PC = Program Counter

## Jump-and-link Instructions

| Bit position | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| J-type | [20] | imm[10:1] | | | | | | | | | | [11] | imm[19:12] | | | | | | | | rd | | | | | opcode | | | | | | |
| jalr – I-type | imm[11:0] | | | | | | | | | | | | rs1 | | | | | 3'b000 | | | rd | | | | | opcode | | | | | | |

| opcode | instruction | description | operation |
|---|---|---|---|
| 7'h67 | `jalr rd, rs1, imm` | jump and link register | PC = rs1 + sign_ext(imm), rd = PC + 4 |
| 7'h6f | `jal rd, label` | Jump and link | PC = JTA, rd = PC + 4 |

JTA = Jump Target Address, PC = Program Counter

### Register names

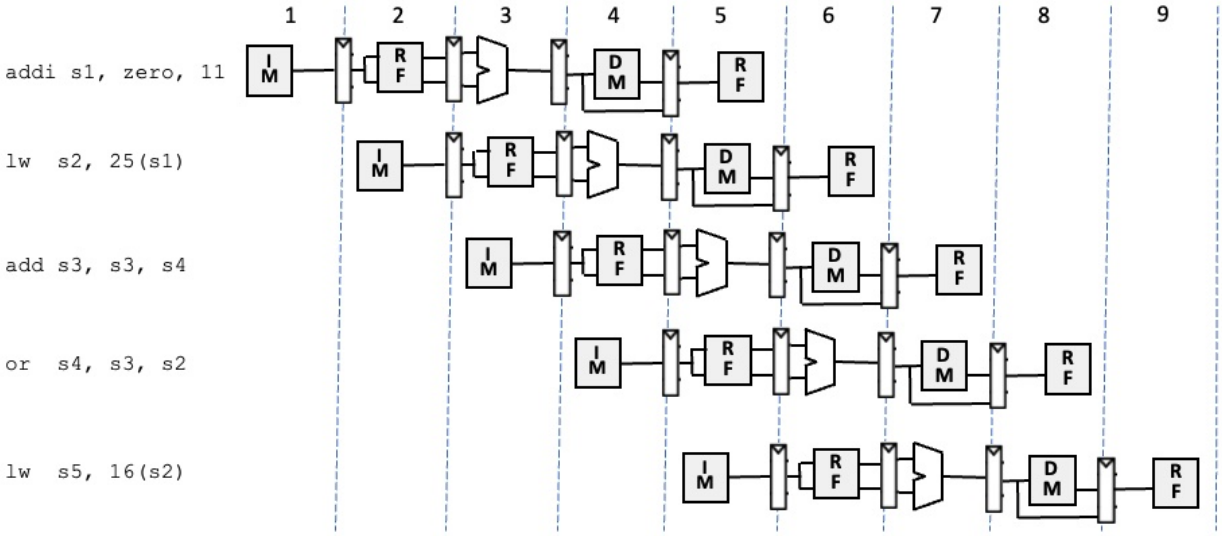
| name | register number | usage |
|---|---|---|
| zero | x0 | Constant value 0 |
| ra | x1 | Return Addrss |
| sp | x2 | Stack Pointer |
| gp | x3 | Global Pointer |
| tp | X4 | Thread Pointer |
| t0 – t2 | x5 – x7 | Temporary registers |
| s0/fp | x8 | Saved register/Frame Pointer |
| s1 | x9 | Saved register |
| a0 – a1 | x10 – x11 | Function arguments/ return values |
| a2 – a7 | x12 – x17 | Function arguments |
| s2 – s11 | x18 – x27 | Saved registers |
| t3 – t6 | x28 – x31 | Temporary registers |

### Abbreviations

| | |
|---|---|
| imm | 12-bit signed immediate ($imm_{11:0}$) |
| uimm | 5-bit unsigned immediate ($imm_{4:0}$) |
| upimm | 20-bit upper immediate ($imm_{31:12}$) |
| address | memory addres: rs1 + sign_ext($imm_{11:0}$) |
| [address] | data at memory location address |
| BTA | branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0}) |
| JTA | jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0}) |
| label | text label indicating instruction address |
| sign_ext | value sign-extended to 32 bits |
| zero_ext | value zero-extended to 32 bits |

SUPPLEMENTARY SHEET FOR QUESTIONS 4 & 5

You may tear off this page, annotate it with your answer, and attach it to the answer book.




| V | tag | data | set |
|---|---|---|---|
| | | | 111 |
| | | | 110 |
| | | | 101 |
| | | | 100 |
| | | | 011 |
| | | | 010 |
| | | | 001 |
| | | | 000 |

Archive

This page intentionally left blank.

Answers

SUPPLEMENTARY SHEET FOR QUESTION 8

You may tear off this page, annotate it with your answer, and attach it to the answer book.

Diagram for question 8d:

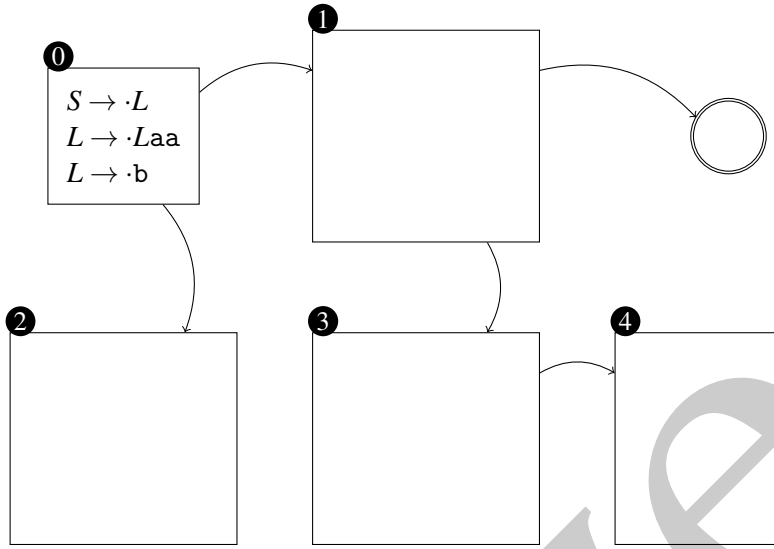



Table for question 8e:

| **STATE** | **ACTION** | | | **GOTO** |
|---|---|---|---|---|
| | a | b | $ | $L$ |
| 0 | | | | |
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |

Answers